

Datathon 2026: Robinreal

Hybrid Listing Search & Ranking

Team AlphaXAM

Adhithya Laxman Ravi Shankar Geetha Aulia Kharis Rakhmasari
Mattia Loszach Xander Yap

Swiss Real-Estate Search Challenge — Technical Overview

Abstract

We present a **production-style hybrid retrieval and ranking** pipeline for Swiss real-estate listings. Natural-language queries are mapped to **structured hard filters** and **soft preferences** using a large language model (LLM), with conservative fallbacks when cloud inference is unavailable. Candidates are drawn from **SQLite**, then scored with **dense multilingual embeddings**, **BM25** lexical retrieval, **SigLIP2** visual similarity over precomputed image shards, **geospatial proximity** to points of interest, **language matches**, and **soft price-band** intent. Per-query **LLM-emitted weights** blend these signals. We ship a **FastAPI** service exposing `POST /listings` for evaluation, plus an optional **Model Context Protocol (MCP)** server and a **React** map/list widget. Runtime artifacts (database, BM25 index, geo caches, VLM shards) are synchronized from **Amazon S3**; listing photos may be served from S3 or augmented when CDN links expire.

1 Problem and Objectives

Real users mix **strict constraints** (city, price ceiling, minimum rooms) with **soft preferences** (“bright”, “quiet”, “near ETH”, “family-friendly”). A strong system must:

1. **not** return listings that violate must-have constraints,
2. rank the remainder by overall relevance, and
3. handle **vague language**, **missing fields**, and **multilingual** input (DE/FR/IT/EN are common in Switzerland).

Our objective is **Challenge 1**: improve empirical search quality on the Robinreal listing corpus using the provided harness, while keeping the ranking stack **inspectable** (per-signal scores and reasons) and **deployable** (single-container workflow via Docker Compose, optional GPU profile for local embedding throughput).

2 Architecture Overview

The logical data flow proceeds as follows:

User query → **Query understanding (LLM)** → **SQL HardFilters** → **Quality post-filter** (non-offers, geo sanity) → **Parallel relevance signals** (dense, BM25, VLM, geo, language, price) → **Weighted fusion** → **Ranked list with explanations**; optional **image URL enrichment** for the HTTP response.

The MCP layer (`apps_sdk/server`) calls the same search API over HTTP and embeds a static-built Vite widget for ChatGPT/Claude-style clients. A separate **POI tool** returns nearby transit, supermarkets, schools, or universities for a given coordinate pair, using the same Overpass-derived caches as the ranker’s geo features.

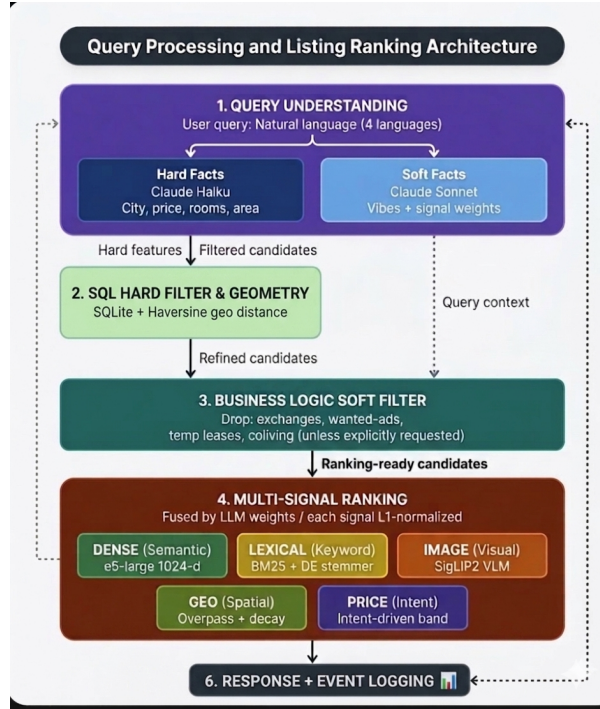


Figure 1: Algorithm Pipeline

3 End-to-End Pipeline

3.1 Query Understanding

A **single** LLM invocation (Anthropic Claude via the **Anthropic API**, or a **rule-based** extractor if both fail) maps the utterance into a typed `QueryUnderstanding`: `HardFilters`, `SoftPreferences`, `soft.weights` (per downstream signal), optional **anchors** (e.g. ETH Zentrum, Bahnhof), **locality hints** (Kreis, suburb names—distinct from coarse *city*), language, and an English interpretation string.

The system prompt encodes Swiss rental-market rules; for example, mapping English “*N bedrooms*” to Swiss **min rooms** = $N+1$ (the living room counts as a *Zimmer*), and separating **must-have** balcony from **nice-to-have** parking.

3.2 Listing Quality (Post-SQL)

`listing-quality.py` removes listings that pass numeric filters but are **not actual offers**: apartment **swap** posts, “wanted”/seeker ads, and listings clearly outside Switzerland when the query targets CH. For queries that imply **long-term** tenancy, short-term/**befristet** leases can be down-ranked or filtered. This addresses failure modes where lexical similarity to the user’s wish-list is high but the document type is wrong.

3.3 Relevance Signals (Hybrid)

Each surviving listing receives partial scores in $[0, 1]$ (normalized before fusion):

- **Dense** — Embedding similarity between query and listing text fields; embeddings are stored in SQLite (Bedrock Cohere) or computed with **fastembed** (e.g. multilingual E5/BGE-M3) when `USE_LOCAL_EMBEDDINGS=1`.
- **BM25** — Multilingual BM25 with stemming and **language detection** per document/query for robust lexical matching on descriptions and titles.
- **VLM** — Offline **SigLIP2** image embeddings grouped in `shard*.npz` files; at query time the **text tower** encodes the user query and scores against shard rows when the query contains **visual** cues (brightness, modern kitchen, view, etc.).

- **Geo** — Distances in meters to nearest transit, supermarket, school; **anchor** proximity (e.g. university column for campus-style queries). Weights default to zero unless soft intent enables them (e.g. “near schools” activates school distance).
- **Language Match** — Alignment in query and description language should be prioritized as easier communication with landlords usually improves quality of life.
- **Price band** — Soft alignment between stated price intent and listing rent/price when the query expresses budget descriptively (“affordable”) rather than only as a hard cap.

3.4 Fusion

Default fusion weights exist for dense/BM25/VLM/geo/price; the LLM’s `soft.weights` are **remapped** onto internal signal names (`dense_text`→`dense`, `clip_image`→`vlm`, etc.) and combined with the per-listing normalized scores. The final `score` and a short `reason` string (top contributing signals) accompany each hit for transparency.

3.5 Response Enrichment

The listings route can rewrite **image URLs** to **S3 HTTPS** paths or API-static paths when CDN links are stale, using `platform_id` and bucket listing (`app/core/s3.py`), subject to environment flags.

4 Ranking Signal Defaults

Signal	Default weight	Gated by intent?
Dense	0.35	Always on
BM25	0.20	Always on
VLM	0.20	Yes (visual language)
Geo (transit)	0.10	Yes (<code>near_public_transport</code> , etc.)
Geo (school)	0.05	Yes (family / schools)
Geo (anchor)	0.15	Yes (named landmarks)
Language Match	0.05	Yes (different query and description language)
Price band	0.10	Yes (soft price intent)

Table 1: Baseline weights; the LLM may override via `soft.weights` per query.

5 Data and Infrastructure

Artifacts bucket.

Layer	Components
API	FastAPI, Uvicorn, Pydantic schemas
Storage	SQLite (<code>listings.db</code>), pickle BM25, NPZ VLM shards
Embeddings	AWS Bedrock Cohere <i>or</i> fastembed (CPU/GPU extra)
LLM	Claude Sonnet family (Bedrock / direct API)
Lexical	rank-bm25, snowball stemmers
Vision	PyTorch, Transformers, SigLIP2-so400m
MCP	FastMCP, streamable HTTP, widget HTML resource
Frontend	React, Vite, MapLibre GL
Cloud	AWS S3 (artifacts + crawl images), boto3, IAM credentials
Containers	Docker Compose; bootstrap job pulls S3 before <code>api/mcp</code>

Table 2: Components per architectural layer.

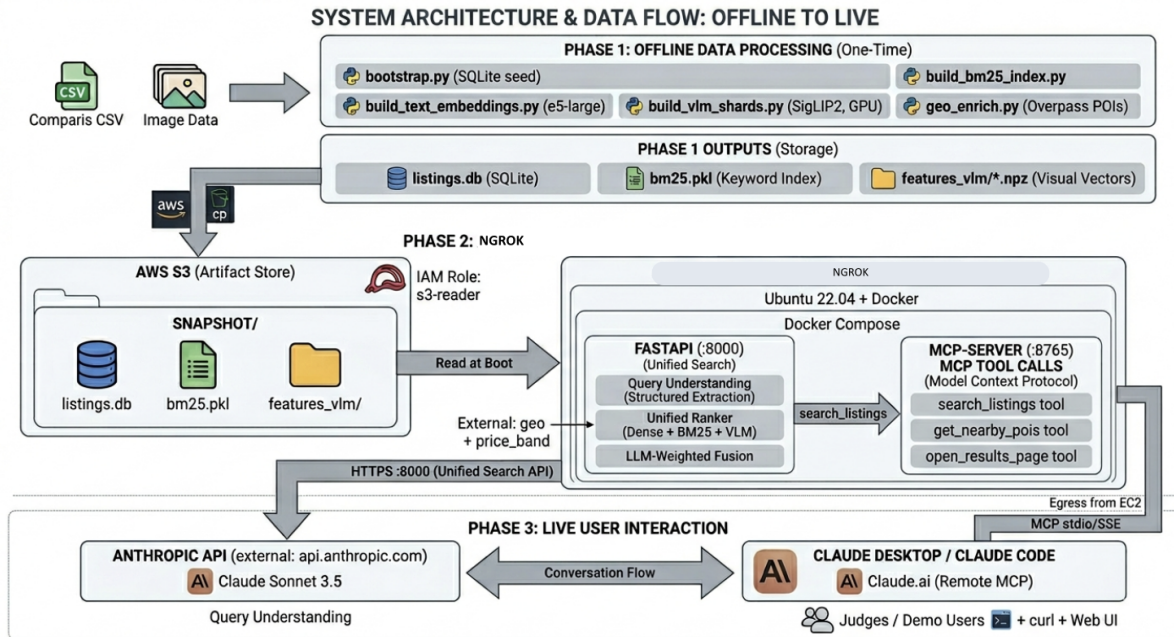


Figure 2: Tech Stack

On startup, we sync: Vector embeddings store, lexical search index, geospatial metadata and CLIP feature shards. Optional raw/downloads/ syncs tens of thousands of image files for offline inspection or re-embedding (disabled by default).

Crawl image bucket. Challenge images follow the prod/<source>/images/platform_id=... layout; the API can list objects to build fresh URLs when marketing CDNs return 404.

Session events (bonus path). session_events in SQLite can log clicks, dwell, and saves; session_id may be threaded into search for future personalization without changing the core evaluation contract.

6 MCP and Demo UI

The MCP server exposes:

- **search_listings** — returns structured JSON for the model and a **widget resource URI** for the UI;
- **get_nearby_pois** — accepts lat/lon plus a category.

The widget bundle is built with `npm run build`, served under `/widget-assets`, and referenced via `APPS.SDK.PUBLIC.BASE_URL` for remote HTTPS clients. Content-Security-Policy metadata whitelists map tile CDNs, Comparis asset hosts, S3 image origins, and optionally the public API origin for `/raw-data-images`.

7 API Contract (Evaluation)

POST /listings — Input: query, limit, offset. Output: ListingsResponse containing meta and a list of RankedListingResult objects, each with listing_id, fused score, human-readable reason, and a nested listing record (title, geo, price, rooms, features, image URLs, etc.).

GET /health — liveness probe.

8 Design Challenges Addressed

- **Ambiguous adjectives** — routed to soft scores and VLM/geo rather than brittle SQL.
- **Conflicting soft goals** — the LLM assigns relative weights; fusion produces a single ordering with explainable dominant signals.

- **Incomplete listings** — degraded signals yield zeros, not crashes; relaxation avoids empty result sets when possible.
- **Multilingual and Swiss conventions** — prompt rules and room-count handling reduce systematic mismatches.

9 Limitations and Future Work

Commute times. SBB/ZVV isochrones are not fully wired as hard constraints in all paths — geo currently uses precomputed POI distances and anchors.

Personalization. Bonus personalization beyond event logging would require a training or bandit layer on top of `session_events`.

Latency. First SigLIP2/HF download on cold instances can take minutes without a baked cache volume.

10 External Services

Anthropic Claude; Amazon Bedrock (Claude + Cohere embeddings); Anthropic direct API as fallback; Hugging Face model hubs for SigLIP2 and fastembed weights; optional Cloudflare tunnel or reverse proxy for public HTTPS demos.

11 Reproducibility

Clone the repository, copy `.env.example` to `.env`, run `uv sync`, then execute `docker compose up --build` (set `BOOTSTRAP_FORCE=1` to refresh artifacts).

Core ranking logic lives in `app/participant/unified_ranker.py`; the query schema is in `app/participant/schemas.py`. Benchmarks and LLM-judge result dumps may live under `results/eval/` for paper-style discussion.